Snake++ · LLMs course project

Fast-track work plan: version-aware RAG coding assistant

Team: Secara Lucas · Kevin Petras · Enea Sebastian · Dolha Roberto

Prepared 25 October 2026 · Plan runs Monday 26 October to Sunday 22 November 2026

Goal. Build the full system from the RFC and finish the experiments in **3 weeks** (26 Oct to 15 Nov), then use week 4 (16 to 22 Nov) only for the report, slides and buffer. We go fast by fixing the interfaces on day 1 so all four people work in parallel from the start, and by letting Claude, Claude Code and Copilot write first drafts of almost all code and text while people review, test and decide.

Section	What it covers
1. How we go fast	Working principles, scope-cut order
2. Roles	Who owns what
3. AI toolkit	Which AI tool to use for which job, and the rules for using it
4. Day 0 setup	Repo layout, CLAUDE.md, environments, shared interfaces
5. Timeline and milestones	Week-by-week plan with gates
6 to 9. Personal plans	Day-by-day tasks, deliverables and ready-made Claude Code prompts for each person
10. Experiment runbook	The six configurations and how to run them
11. Report and demo	Who writes which section
12. Risks and fallbacks	What to do when something breaks
13. Routine and checklist	Daily sync, PR rules, final deliverables

Dates assume no other course deadlines get in the way. If your course deadline is earlier or later, shift the whole calendar; the order of work stays the same.

1. How we go fast

- **Interfaces before code.** On day 0 we agree on four data formats (chunk, retrieval result, generation, evaluation result). After that, nobody waits for anybody: each person works against stub data that follows the format.
- **Thin end-to-end first.** By Sunday 1 November the whole pipeline must run on a tiny slice (pandas docs only, about 20k Stack Overflow posts, the pandas part of DS-1000). We widen it only after it works.
- **AI writes, humans verify.** Claude Code writes the first version of every module and its tests. The owner reads the code, runs it and is responsible for it. No PR is merged without passing tests and one human review.
- **Cache everything.** Embeddings, retrieval results and model generations are saved to disk keyed by config, so a rerun never repeats expensive work.
- **Decide in minutes.** Small decisions go to the owner of that area. Only interface changes need the whole team.
- **Small daily goals.** Each person posts what they will finish today and what blocks them (section 13).

Scope-cut order

If we fall behind, we cut in this order and keep the core study intact:

#	Cut	Effect
1	Hugging Face extension (transformers/datasets corpus)	Version-sensitive set keeps only pandas 1.x vs 2.x questions
2	MBPP	HumanEval alone stays as the general-coding control
3	Second embedding model	One embedding model only
4	SciPy in corpus and DS-1000	Four libraries instead of five
5	Stack Overflow down to about 50k posts	Less coverage, faster indexing
Never cut	No-retrieval baseline, hybrid + rerank, version filter, decontamination	These are the RFC's main claims

2. Roles

Each person owns one area end to end, including its tests and its section of the final report. Kevin also owns the repository and integration, because retrieval sits in the middle of the pipeline.

Person	Role	Owns	Main outputs
Secara Lucas	Data and corpus	Docs and Stack Overflow ingestion, chunking, version tags, decontamination, licensing metadata	Chunk files for every library, corpus stats, decontamination report
Kevin Petras	Retrieval and integration lead	Repo, CI, interfaces; BM25, dense index, fusion, reranker, version filter; pipeline wiring	Retriever package, retrieval metrics, pipeline configs, integration
Enea Sebastian	Generation and evaluation	Docker sandbox, DS-1000/HumanEval/MBPP harness, generator wrapper, prompts, citations, pass@1	Eval harness, all experiment runs, results files

Person	Role	Owns	Main outputs
Dolha Roberto	Benchmarks, analysis, demo, report	Version-sensitive question set, statistics, plots, failure analysis, demo app, report and slides	Benchmark, analysis notebook, demo, report and slides

Pairing. Lucas and Kevin pair on the chunk format and the version tags (they both depend on it). Enea and Roberto pair on test execution, because the version-sensitive set runs on Enea's sandbox. Every PR is reviewed by the pair partner first.

3. AI toolkit

Tool	Use it for	Who
Claude Code (terminal, VS Code, or the web app)	Writing modules and tests from the specs in this plan, refactoring, debugging failing runs, writing scripts that batch experiments, reading big logs	Everyone, daily
Claude (chat, this Project)	Planning, design questions, summarizing papers, drafting report sections, reviewing PR diffs, explaining errors. Keep the RFC and this plan in the Project so every chat has context	Everyone
GitHub Copilot	Line-level completion while you edit by hand	Everyone, optional
Claude Code GitHub review or a Claude chat with the diff	A second review of each PR before the human review	PR author
Claude for data drafts	Drafting version-sensitive questions from changelogs, writing test stubs, labelling failure categories	Roberto, Lucas

Rules for using AI

- **Every AI-written function gets a test you have run.** If you cannot explain what a piece of code does, do not merge it.
- **Never let AI produce ground truth without execution.** Benchmark questions, expected outputs and gold documents are only kept if a real test run confirms them.
- **Keep the AI away from the test sets during development.** Don't paste DS-1000 or HumanEval solutions into prompts that shape the pipeline. This keeps the evaluation honest.
- **The generator model inside our system is fixed.** Claude helps us build the system; it is not the model being evaluated unless we decide that explicitly and keep it fixed across all six configs.
- **No secrets in prompts or the repo.** API keys go in a local `.env` that is git-ignored.
- **Check the course rules on AI use** and add a short AI-use statement to the report saying which tools we used and for what.

Working loop with Claude Code

```
1. git checkout -b feat/<area>-<task>
2. Start Claude Code in the repo. It reads CLAUDE.md automatically.
3. Paste the task prompt from your section of this plan (edit details as needed).
4. Ask it to write tests first, then the code, then to run the tests.
5. Read the diff yourself. Run it on real data once.
6. Open a PR. Ask Claude to review the diff. Then your pair partner reviews.
7. Merge when CI is green.
```

4. Day 0 setup (Monday 26 October)

Everyone does the setup tasks below in the first session, ideally together on a call. Target: done by Monday evening.

When	Task	Done when
Mon AM	**Kevin:** create GitHub repo, the folder layout below, `pyproject.toml`, ruff + pytest, GitHub Actions CI, CLAUDE.md, issue board with every task in this plan	Everyone can clone, install and run `pytest`
Mon AM	**Lucas:** request the Stack Overflow data dump from the account settings page and start the download now (it is large). Check the community list of watermarked dumps	Download running
Mon AM	**Enea:** find out which library versions DS-1000's official execution environment pins, and post them to the team	Versions posted; Lucas indexes docs for those versions
Mon PM	**All:** agree on and freeze the interfaces in `snakepp/schemas.py` (below)	Merged to main
Mon PM	**Enea:** decide the generator: an open 7B code model on Kaggle/Colab GPU through vLLM, or an API model. Measure one DS-1000 problem end to end	Decision and time/cost per problem written in README
Mon PM	**Roberto:** put RFC, this plan and key papers (CodeRAG-Bench, DS-1000) in the Claude Project; ask Claude for a related-work outline	Related-work outline in docs/

Repository layout

```
snakepp/
  CLAUDE.md                # conventions Claude Code must follow (see below)
  configs/                 # one YAML per experiment configuration
  data/                    # git-ignored; raw/, chunks/, indexes/, cache/
  snakepp/
    schemas.py             # the frozen interfaces (pydantic)
    corpus/                # Lucas: docs.py, stackoverflow.py, versions.py, decontam.py
    retrieval/             # Kevin: bm25.py, dense.py, fusion.py, rerank.py, version_filter.py
    generation/            # Enea: llm.py, prompts.py, citations.py
    evaluation/            # Enea: sandbox/, ds1000.py, humaneval.py, mbpp.py, metrics.py
    benchmarks/version_set/# Roberto: questions + tests for pandas 1.x/2.x and HF
    pipeline.py            # Kevin: config -> retrieve -> generate -> evaluate
  analysis/                # Roberto: notebooks, plots
  demo/                    # Roberto: Gradio app
  tests/                   # unit tests for every module
  report/                  # LaTeX or Markdown report and slides
```

CLAUDE.md (starting version)

```
# Snake++ conventions
- Python 3.11. Format with ruff. Type hints everywhere. Pydantic models from snakepp/schemas.py.
- Never change schemas.py without a team-approved issue.
- Every new function gets a pytest test in tests/. Run `pytest -q` before saying a task is done.
- Read and write data only under data/ (git-ignored). Large files are never committed.
- Anything slow (embedding, LLM calls, test execution) must be cached in data/cache/
  keyed by a hash of its inputs and config.
- Never read DS-1000, HumanEval or MBPP solutions while building the pipeline.
- Keep functions small; one module per concern; CLI entry points with typer.
```

Frozen interfaces

```
class Chunk(BaseModel):
    id: str                    # stable hash of source + position
    text: str
    source: Literal["docs", "stackoverflow"]
    library: str               # "pandas", "numpy", ...
    version: str | None        # "2.2.2" for docs; inferred for SO
    version_inferred: bool     # True when derived from post date
    url: str
    title: str
    score: int | None          # SO answer score
    created: date | None
    license: str               # "CC BY-SA 4.0", "BSD-3-Clause", ...

class Retrieved(BaseModel):
    chunk_id: str; rank: int; score: float; stage: Literal["bm25","dense","fused","reranked"]

class Generation(BaseModel):
    task_id: str; config: str; prompt_hash: str; code: str
    cited_chunk_ids: list[str]; raw_output: str; latency_s: float

class EvalResult(BaseModel):
    task_id: str; benchmark: str; config: str; passed: bool
    error_type: str | None; stderr_tail: str | None
```

5. Timeline and milestones

Week	Focus	Milestone gate (end of week)
Week 1 26 Oct to 1 Nov	Setup, thin slice of every component, first end-to-end run	**M1 (Thu 29 Oct):** no-retrieval baseline scored on DS-1000 pandas subset; pandas docs chunked; BM25 works. **M2 (Sun 1 Nov):** hybrid retrieval to generation to tests runs end to end on the pandas slice
Week 2 2 to 8 Nov	Full corpus, decontamination, reranker, version filter, version-sensitive set drafted	**M3 (Sun 8 Nov):** all six configurations run once on the full DS-1000 subset (5 libraries) and HumanEval; first results table
Week 3 9 to 15 Nov	Version-sensitive evaluation, final runs, analysis, demo	**M4 (Sun 15 Nov):** final numbers frozen, plots done, failure analysis done, demo recorded
Week 4 16 to 22 Nov	Report, slides, buffer for anything late	**M5 (Sun 22 Nov):** report and slides submitted; repo tagged v1.0 with a reproduction README

Dependency map

- Enea's DS-1000 version list (Mon) → Lucas indexes those doc versions.
- Lucas's first chunk file (Wed 28 Oct) → Kevin builds real indexes (until then, Kevin uses 200 hand-made stub chunks).
- Kevin's `retrieve()` (Thu 29 Oct) → Enea's retrieval-augmented prompts.
- Enea's sandbox (Wed 28 Oct) → Roberto runs the version-sensitive tests.
- Lucas's decontamination (Wed 4 Nov) → only the clean index is used for the final runs.
- All final results (Sat 14 Nov) → Roberto's analysis and plots.

6. Secara Lucas: data and corpus

You produce every chunk the system can retrieve. The quality of your version tags decides whether our main contribution works, and your decontamination decides whether our results can be trusted.

Day-by-day tasks

When	Task	Done when
Mon 26	Start the Stack Overflow dump download (account settings → Data dump access; tick the box that confirms no LLM training, which fits our retrieval use). Check the watermark list for the dump you get	Download running; note which dump date we use
Mon 26 to Tue 27	Docs ingestion for pandas at the DS-1000 version and at 2.2.x: clone repo at release tag, parse `doc/source/**/*.rst` with docutils, split by heading, keep code blocks whole, ~300 to 800 tokens per chunk	`data/chunks/docs_pandas_*.parquet` with valid Chunk rows
Tue 27 to Wed 28	Stack Overflow streaming parser: iterate `Posts.xml` with lxml iterparse, keep questions tagged python + one of our libraries, accepted answer, score ≥ 3; join question + accepted answer into one chunk; strip HTML, keep code	First 20k posts chunked (needed for M2)
Wed 28	Version inference for SO: get release dates for every library from the PyPI JSON API, map post date to the version current then; override when the text names a version	`versions.py` + tests; 50 spot checks by hand
Thu 29 to Sat 31	Docs for NumPy, Matplotlib, scikit-learn, SciPy, Python stdlib (and transformers, datasets if not cut). Use the same parser; add markdown parsing for HF docs	All doc chunk files + stats table (chunks per library/version)
Sun 1 to Tue 3	Full SO subset: 50k to 200k posts across our tags	Full SO chunk file + stats
Tue 3 to Wed 4	Decontamination: MinHash (datasketch, 5-gram shingles) and embedding cosine between DS-1000 prompts and SO chunks; remove pairs above threshold; build a *leaky* and a *clean* variant	Report: thresholds, number removed, 20 examples checked by hand
Thu 5 to Sun 8	Help Kevin with chunking tweaks after first retrieval metrics; attribution data (author, link, license) for every SO chunk	Attribution columns filled
Week 3	Write report sections: Data, Version tagging, Decontamination; produce corpus figures	Sections in report/

Deliverables

- Chunk parquet files for every library and version, plus the leaky and clean SO variants
- Corpus statistics table (chunks, tokens, per library, per version, per source)
- Decontamination report with examples
- Report sections on data, version tagging, decontamination and licensing

Ready-made Claude Code prompts

Docs parser

```
Read CLAUDE.md and snakepp/schemas.py. Write snakepp/corpus/docs.py that takes a
path to a Sphinx docs folder, a library name and a version, parses every .rst file
with docutils, splits by section heading, never splits a code block, merges tiny
sections, and yields Chunk objects with stable ids. Add a typer CLI that writes a
parquet file. Write tests with a small fake .rst tree first, then run them.
```

Stack Overflow parser

```
Write snakepp/corpus/stackoverflow.py. Stream a Posts.xml file with lxml iterparse
(memory must stay flat). Keep questions whose tags include python and any of
[pandas, numpy, matplotlib, scikit-learn, scipy], with an accepted answer and
score >= 3. Join question title, body and accepted answer into one Chunk, convert
HTML to text but keep <code> blocks as fenced code. Store url, score, created date,
license by date. Test on a 30-post fixture XML.
```

Decontamination

```
Write snakepp/corpus/decontam.py. Given a list of benchmark prompts and the SO chunk
file, find near-duplicates with datasketch MinHash LSH (5-word shingles) and, for
candidates, embedding cosine similarity. Output a CSV of matches with scores and a
clean parquet without them. Make thresholds CLI options. Add tests with planted
duplicates.
```

7. Kevin Petras: retrieval and integration lead

You own the repository and the middle of the pipeline. Your first job is unblocking the others on day 0; after that you build the retrieval stack and wire every part together through configs.

Day-by-day tasks

When	Task	Done when
Mon 26	Repo, CI, CLAUDE.md, schemas.py, issue board with every task from this plan (section 4)	Team can clone, install, test
Tue 27	200 stub chunks (hand-written or generated from pandas docs pages) so retrieval work can start before Lucas's data lands	`tests/fixtures/stub_chunks.parquet`
Tue 27 to Wed 28	BM25 with an identifier-aware tokenizer: split on `. _ ( )` and camelCase, keep the full identifier too (`DataFrame.groupby` → dataframe, groupby, dataframe.groupby)	Tests show exact identifier queries rank the right chunk first
Wed 28 to Thu 29	Dense retrieval: sentence-transformers (start with bge-small-en-v1.5 or e5-small-v2), FAISS IndexFlatIP, embeddings cached to disk; common `retrieve(query, k, config)` API	M1: retrieve() works on real pandas chunks
Fri 30 to Sun 1	Reciprocal rank fusion; `pipeline.py` wiring config → retrieve → Enea's generate → evaluate; YAML configs	M2: end-to-end run on pandas slice
Mon 2 to Tue 3	Cross-encoder reranker on top 20 to 50 (bge-reranker-base or ms-marco-MiniLM-L-6-v2), batched, cached	Reranker on/off by config
Wed 4 to Thu 5	Version filter: hard mode (drop mismatches) and soft mode (multiply score by a penalty); version detection from query text, explicit argument or `importlib.metadata`	Both modes tested; soft is default
Fri 6 to Sun 8	Retrieval metrics: recall@k and MRR on tasks with gold documents (check which CodeRAG-Bench tasks have them); tune k, fusion constant, penalty on a dev split, not on the test tasks	M3: six configs run; retrieval metrics table
Week 3	Final runs support, freezing code, tagging release, reproduction README; report sections on retrieval and version filtering	v1.0 tag, report sections

Deliverables

- snakepp/retrieval package with BM25, dense, fusion, reranker, version filter and tests
- pipeline.py and the six YAML configs (plus version-filter variants)
- Retrieval metrics table (recall@k, MRR) and tuning notes
- Reproduction README; report sections on retrieval and integration

Ready-made Claude Code prompts

BM25 with code-aware tokenizer

```
Read CLAUDE.md and schemas.py. Write snakepp/retrieval/bm25.py using rank_bm25.
Tokenizer: lowercase, keep whole dotted identifiers AND split them on '.', '_',
parentheses and camelCase. Build from a chunk parquet, save/load with pickle under
data/indexes/. Method search(query, k) -> list[Retrieved]. Tests: a query
'DataFrame.append removed' must rank the chunk containing DataFrame.append first.
```

Dense index

```
Write snakepp/retrieval/dense.py: embed chunks with sentence-transformers (model name
from config), normalize, store in FAISS IndexFlatIP with an id map, cache embeddings
as .npy keyed by model + chunk-file hash. search(query, k) -> list[Retrieved]. Add a
batch size option and a progress bar. Test with the stub chunk fixture.
```

Fusion, rerank, version filter

```
Write fusion.py (reciprocal rank fusion, constant from config), rerank.py
(CrossEncoder, rerank top-n, cached by query+chunk ids) and version_filter.py with
hard and soft modes using packaging.version. Then write snakepp/retrieval/__init__.py
exposing retrieve(query, target_versions, config). Tests for each part.
```

8. Enea Sebastian: generation and evaluation

You build the part that turns retrieved chunks into code and proves whether the code is right. Every number in the report comes out of your harness, so it must be reproducible and safe.

Day-by-day tasks

When	Task	Done when
Mon 26	Find DS-1000's pinned library versions and post them. Choose generator (open 7B code model via vLLM on Kaggle/Colab GPU, or API) and time one problem	Versions posted; generator decision in README
Mon 26 to Wed 28	Docker sandbox: image with the DS-1000 pinned versions; run code with timeout, no network, memory limit; return pass/fail, error type, stderr tail	Sandbox runs a known-good and known-bad snippet correctly
Tue 27 to Thu 29	DS-1000 loader limited to Pandas, NumPy, Matplotlib, Sklearn, Scipy using the official evaluation code; no-retrieval generation; pass@1 computation	M1: no-retrieval baseline score
Thu 29 to Sun 1	Prompt templates: no-context and with-context (numbered chunks, ask for code plus cited chunk numbers); output parser that extracts code and citations	M2: end-to-end run with Kevin
Mon 2 to Tue 3	HumanEval and MBPP runners (official harnesses) in the same sandbox and result format	Both benchmarks score with no retrieval
Wed 4 to Sun 8	Batch runner: all configs × benchmarks, resumable, generations cached; temperature 0 for pass@1; log latency and tokens. Run every config once	M3: first full results table
Mon 9 to Wed 11	Second sandbox image with pandas 1.x and with pandas 2.x / current HF so Roberto's version-sensitive tests can run both ways	Both images build and pass smoke tests
Thu 12 to Sat 14	Final runs on the clean index: all configs, all benchmarks, version-sensitive set; citation accuracy (cited chunk contains the API the code uses)	M4: final results files frozen
Week 4	Report sections: evaluation setup, generator, metrics	Sections in report/

Deliverables

- Sandbox Docker images and the snakepp/evaluation package with tests
- Generator wrapper, prompt templates, citation parser
- Results files (EvalResult JSONL) for every config and benchmark, with the exact commands used
- Report sections on evaluation setup and metrics

Ready-made Claude Code prompts

Sandbox

```
Read CLAUDE.md. Write snakepp/evaluation/sandbox/: a Dockerfile installing the
library versions in versions.txt, and runner.py that copies a code string plus a
test string into a container, runs it with --network none, a 30 s timeout and a
2 GB memory limit, and returns an EvalResult (passed, error_type, stderr_tail).
Run containers in parallel with a worker pool. Test with passing, failing,
timeout and infinite-memory snippets.
```

DS-1000 harness

```
Write snakepp/evaluation/ds1000.py that loads DS-1000 from the official release,
keeps only Pandas, Numpy, Matplotlib, Sklearn and Scipy problems, builds the prompt
for each, sends generated code through the sandbox using the official test code,
and writes EvalResult JSONL. Do not print or store reference solutions in any
prompt. Add a --limit flag for quick runs.
```

Prompts and citations

```
Write snakepp/generation/prompts.py with a no-context template and a context template
listing chunks as [1]..[k] with library and version, asking for one python code
block and a 'Sources: [n]' line. Write citations.py to parse both. Tests with
messy model outputs (missing fences, extra text, no sources).
```

9. Dolha Roberto: benchmarks, analysis, demo and report

You turn the system into evidence and a story. You build the test set that shows version awareness, analyse every result, build the demo and lead the report.

Day-by-day tasks

When	Task	Done when
Mon 26	Claude Project set up with RFC, plan and papers; related-work outline	docs/related_work.md
Tue 27 to Fri 30	Version-sensitive question set, pandas part: collect APIs removed or changed between pandas 1.x and 2.x from the official release notes (append, iteritems, swaplevel arguments, etc.); let Claude draft questions and tests; keep only questions whose tests behave as expected when run	First 15 questions with tests
Sat 31 to Wed 4	HF part (transformers/datasets) if not cut: same method from their release notes and migration guides; target 30 to 50 questions in total	Full set in benchmarks/version_set/ with a README
Thu 5 to Sun 8	Analysis notebook skeleton working on M3 results: results table, bar charts, per-library breakdown, paired bootstrap confidence intervals or McNemar test between configs	Notebook regenerates all figures from results files
Mon 9 to Wed 11	Demo: Gradio app with question box, detected versions, retrieved chunks with tags, generated code and citations; record a 2-minute screen video	demo/ runs locally; video saved
Thu 12 to Sun 15	Failure analysis on final results: sample about 50 failures, let Claude propose labels (retrieval miss, wrong version, ignored context, syntax, logic), check each by hand; plots final	M4: final figures and failure table
Week 4	Report lead: assemble all sections, intro, related work, results, discussion, conclusion; slides; final read-through	M5: report and slides done

Deliverables

- Version-sensitive benchmark (30 to 50 questions, each with tests under both versions)
- Analysis notebook with all tables, figures and significance tests
- Gradio demo and a 2-minute demo video
- Final report and slides (with everyone's sections)

Ready-made Claude Code prompts

Draft version-sensitive questions

```
I will paste the pandas 2.0 release notes section on removals and deprecations.
For each removed or changed API, write: a natural user question that does not name
the API, a correct pandas 2.x solution, and a pytest test that passes on 2.x. Also
note what the old 1.x code was. Output YAML. Do not invent APIs; only use what is
in the pasted text.
```

Analysis notebook

```
Write analysis/results.ipynb that loads every EvalResult JSONL in data/results/,
builds a pass@1 table (config x benchmark x library), draws grouped bar charts with
matplotlib, and computes paired bootstrap 95% confidence intervals for the
difference between each config and the no-retrieval baseline. Save figures to
report/figures/ as PDF.
```

Demo app

```
Write demo/app.py with Gradio: input question and optional library versions; call
snakepp.pipeline.answer(); show detected versions, a table of retrieved chunks
(source, library, version, score, link), the code with syntax highlighting, and
citations linking to the chunk table. Add a 'compare with no retrieval' toggle.
```

10. Experiment runbook

Owner: Enea runs, Kevin maintains configs, Roberto analyses. Every run is a config file plus one command, so anyone can reproduce it.

Config	Retrieval	Corpus	Purpose
c0_none	none	none	Baseline
c1_docs	hybrid + rerank	docs only	Value of documentation
c2_so	hybrid + rerank	Stack Overflow only (clean)	Value of Q&A
c3_dense	dense only	docs + SO	Dense alone
c4_hybrid	BM25 + dense, fused	docs + SO	Value of BM25
c5_full	hybrid + rerank	docs + SO	Full system
c5_full_vf	c5 + soft version filter	docs + SO	Main contribution (version-sensitive set)
c5_leaky	c5 on the leaky SO index	docs + leaky SO	Size of the contamination effect

```
# one configuration on one benchmark
python -m snakepp.pipeline run --config configs/c5_full.yaml --benchmark ds1000 --limit 50
# everything, resumable, cached
python -m snakepp.pipeline sweep --configs configs/*.yaml \
    --benchmarks ds1000 humaneval mbpp version_set
```

- Temperature 0, same generator, same max tokens and same k for every config.
- Results go to `data/results/<config>/<benchmark>.jsonl`; never overwrite final runs, add a date suffix.
- Tune anything (k, penalty, fusion constant) only on a dev split, not on the reported tasks.
- Freeze code and configs on Sat 14 November before the final runs, then tag the commit.

11. Report and demo

Section	Writer	Draft by
Abstract, introduction	Roberto	Mon 16 Nov
Related work	Roberto (Claude summaries, checked against papers)	Sun 8 Nov
Data and corpus, version tagging, decontamination, licenses	Lucas	Mon 16 Nov
Retrieval pipeline and version filter	Kevin	Mon 16 Nov
Evaluation setup, generator, metrics	Enea	Mon 16 Nov
Results, ablation, failure analysis	Roberto with Enea	Wed 18 Nov
Limitations, AI-use statement, conclusion	Kevin	Wed 18 Nov
Slides and demo video	Roberto (everyone presents own part)	Fri 20 Nov
Final proofread, references, reproduction README	All	Sat 21 Nov

Use Claude to turn bullet notes into prose and to check consistency across sections, but every number must be copied from the results files, and every citation checked against the paper.

12. Risks and fallbacks

Risk	Early sign	Fallback	Owner
Stack Overflow dump download is slow or watermarked	Not done by Tue 27	Start with the CodeRAG-Bench released corpora or a community mirror of the same CC BY-SA dump; filter watermarked rows	Lucas
No free GPU time	Generation slower than 30 s/problem	Switch to an API model for generation, cache all outputs, cut MBPP	Enea
DS-1000 environment breaks	Known-good solution fails in sandbox	Use the official DS-1000 Docker setup unchanged	Enea
Retrieval doesn't help	c5 ≤ c0 at M3	Inspect 20 cases; check chunking, k, prompt format; it is still a valid result if analysed honestly	Kevin, Roberto
Version tags too noisy	Spot checks under 80% correct	Use docs-only version filter; treat SO tags as soft hints	Lucas, Kevin
AI-written code hides bugs	Odd numbers, tests that always pass	Planted-bug tests in the sandbox; human review of every PR	All
Person blocked or sick	Task late for 2 days	Pair partner takes over; Claude Code can pick up a well-specified issue quickly	Kevin

13. Routine and final checklist

Daily routine

- **15-minute call every day** (or messages before 10:00): done yesterday, doing today, blocked by.
- Every task is an issue on the board with an owner and a due date from this plan.
- PRs small (under ~400 lines), green CI, reviewed by Claude and then by the pair partner within 12 hours.
- **Sunday review** at the end of each week: check the milestone gate, update the plan, apply scope cuts early if needed.

Final checklist (Sun 22 Nov)

Item	Owner
Repo tagged v1.0; README reproduces every table with listed commands	Kevin
Chunk files and indexes buildable from scripts; licenses and SO attribution documented	Lucas
All results JSONL files and the exact configs used, archived	Enea
Version-sensitive benchmark with README and tests under both versions	Roberto
Analysis notebook regenerates every figure	Roberto
Demo app runs; 2-minute video recorded	Roberto
Report: all sections, AI-use statement, limitations, references checked	All
Slides and rehearsal	All